

miniFlink ex07

Сервис локации и газовых алертов для подземной шахты:
пошаговый разбор архитектуры

Abstract

ex07 — самый большой пример в репозитории miniFlink. Он строит сервис для подземной угольной шахты, который из двух независимых потоков (пакеты от фонарей и пакеты от газовых датчиков) выдаёт три типа выходов: алерты состояния шахтёра, локацию по маякам связи и обогащённые координатами газовые алерты. Под капотом используется почти весь арсенал библиотеки: скользящие окна с `allowed_lateness`, `group_by`, FSM через `process_keyed`, объединение потоков с выравниванием `watermark`, `retract`-семантика для обновлений.

Эта статья — подробный разбор того, *зачем* каждое решение принято таким, как оно принято: какие альтернативы рассматривались, почему отброшены, что бы случилось при наивной реализации. Цель — дать инженеру, выбирающему stream processing, конкретный материал для оценки, а контрибьютору — понимание design-rationale.

Contents

1	Задача	2
1.1	Что должен делать сервис	2
1.2	Что в этой задаче сложного	3
1.3	Почему через библиотеку, а не «просто кодом»	3
1.4	Что есть готового в miniFlink	5
1.5	Что увидим дальше	5
2	Входы	5
2.1	Два независимых потока, а не один	6
2.2	Структура RSSI-пакета	6
2.3	Структура газового пакета	7
2.4	Дрожание канала	7
2.5	Large_mine: масштабная конфигурация	7
3	Три пайплайна	8
3.1	connectivity_alerts: три слоя FSM	8
3.1.1	Слой 1: Last_seen	9
3.1.2	Слой 2: Fsm	9
3.1.3	Слой 3: Self_timer	10
3.1.4	Склейка через process_keyed	10
3.2	median_rssi: скользящие окна и интерполяция координат	11
3.2.1	Шаги пайплайна	11
3.2.2	Линейная интерполяция координат	13
3.2.3	Single-beacon fallback	14
3.2.4	Собирая всё вместе	15
3.3	gas_alerts: три потока и retract вручную	15

3.3.1	Что хотим	15
3.3.2	Почему не window join	16
3.3.3	Три потока на входе	16
3.3.4	Состояние per-шахтёр	17
3.3.5	Шесть случаев на одном газовом пакете	17
3.3.6	Refresh при обновлении позиции	18
3.3.7	Почему руками, а не process_keyed	19
3.3.8	Главный сценарий: позиция приходит позже газа	20
4	Retract как сквозной механизм	20
5	Производительность	20
6	Выводы	20

1 Задача

Представьте себе подземную угольную шахту: километры тоннелей на нескольких горизонтах, в среднем по сорок-сто шахтёров на смену, у каждого фонарь, который раз в пятнадцать секунд шлёт радиопакет с показаниями нескольких ближайших маяков связи, а раз в десять секунд — отдельный пакет с показаниями газовых сенсоров. Эти пакеты летят в диспетчерскую через смесь UDP-радио и LTE-шлюзов, доходят с задержками, иногда дублируются, иногда теряются.

В диспетчерской на стене висит экран с картой шахты и состоянием смены: где находится каждый шахтёр, у кого тревога по газу, кто прижал кнопку SOS, у кого вот-вот сядет батарейка фонаря. Этот экран обслуживает наш сервис.

1.1 Что должен делать сервис

Если разложить требования *диспетчера* на конкретные пайплайны, получается три независимых подсистемы:

Контроль состояния шахтёра. Из RSSI-пакета извлекаем не показания маяков (это работа другой подсистемы), а *косвенные факты*: пакет вообще пришёл, в нём были показания, фонарь двигался, напряжение батареи в норме. По каждому из этих фактов держим независимый таймер и эмитим соответствующий алерт когда таймер перевалил порог: “не было пакетов больше двух минут”, “пакеты идут, но маяков не слышит больше пяти”, “не двигался дольше порога”, “батарея ниже 3,5 В с дребезгом по гистерезису”. Плюс импульс SOS (нажатая кнопка) — мгновенный без таймера.

Локация по маякам. Для каждого шахтёра в скользящем 60-секундном окне с шагом 5 секунд берём топ-2 маяков с самой сильной медианной RSSI и из них приближённо вычисляем координаты шахтёра через интерполяцию между двумя маяками пропорционально расстоянию. Окно скользит каждые 5 секунд, поэтому диспетчерский экран обновляется не реже чем раз в 5 с; при опаздавшем пакете окно *переоткрывается* и эмитит **Retract** старого значения с последующим обновлённым **Data** — так картинка остаётся консистентной.

Газовые алерты с обогащением координатами. Газовый пакет приходит *отдельно* и сам по себе не несёт ID маяков или позиции. Когда газ превышает порог — алерт идёт немедленно (безопасность важнее красоты), но без координат, если их ещё не было. Как только локация для этого шахтёра становится известна (хотя бы по одному маяку), алерт автоматически *обновляется*: старая версия без координат retract’ится,

новая с координатами эмитится. То же самое при смене уровня опасности (Warning → Critical) или существенном изменении ppm. Когда газ возвращается в норму — алерт снимается через `Gas_resolved`.

Эти три подсистемы независимы по бизнес-логике, но обмениваются данными: газовый пайплайн *потребляет* результат пайплайна локации, чтобы обогатить свои алерты координатами.

1.2 Что в этой задаче сложного

Если бы это была учебная задача из курса по базам данных, её можно было бы решить за полчаса: батчевый ETL раз в минуту, JOIN по шахтёру, отрисовка в дашборд. Но в реальной шахте такое решение *бесполезно*, и причин этому несколько.

Время событий не равно времени поступления. Пакет от шахтёра M1 с временной меткой `t=88s` может прийти в диспетчерскую в момент `t=210s`: UDP в шахте теряет пакеты, LTE-шлюз делает повторные попытки, на агрегаторе очереди. Если мы будем считать алерты по wall-clock времени поступления, то получим “M1 не двигался последние 2 минуты” в момент когда M1 уже давно дома — его пакет о движении просто опаздывает по дороге. Сервис должен работать по *event-time* (времени самого события), а не по wall-clock.

Идемпотентность. Тот же UDP-механизм даёт дубликаты: пакет может прийти дважды, потому что шлюз ретранслировал его при сомнениях. Если мы будем считать тревогу по второму дублю “независимо”, получим *два* алерта SOS на одно нажатие кнопки. Диспетчер не должен видеть фантомных тревог.

Опоздавшие пакеты ломают окна. Окно `[60s, 120s)` закрылось в 120с, и пайплайн эмитнул финальное top-2 маяков. Но в 145с пришёл опоздавший пакет с `ts=90s` — он *принадлежит* уже закрытому окну. Нужно либо его выбросить (теряем данные), либо переоткрыть окно, пересчитать агрегат и эмитнуть новое значение, помечая старое как недействительное. Второе правильно для диспетчера; первое было бы дешевле, но даёт непредсказуемый дашборд (на одной и той же временной метке могут быть разные значения в разные моменты wall-clock).

Газовый алерт без координат лучше чем нет алерта. Газ начинает выделяться *раньше* чем шахтёр успевает прийти под маяк — особенно в забое, где маяки ставят редко. Если ждать координат прежде чем эмитить тревогу о 120 ppm CO, можно потерять минуты на ожидании. Алерт идёт сразу. Но как только координаты появляются — они должны *подтянуться* к уже существующей тревоге, не порождая дубль и не давая диспетчеру повод закрыть старый алерт и открыть новый вручную.

Объём. Шахта на сорок шахтёров — мелочь. Реальные большие шахты в Кузбассе или Караганде имеют до четырёх тысяч работников в смену. RSSI-пакетов от такой шахты получается порядка 270 ev/s, газовых — ещё 400 ev/s. Это немного по меркам стримов, но *пайплайн должен держать нагрузку с запасом*: пиковые моменты (несколько горящих участков, грозовая помеха в радио, перезапуск шлюза) дают взрывы в десятки тысяч событий в секунду на короткий период. Сервис не должен в эти моменты терять алерты.

1.3 Почему через библиотеку, а не «просто кодом»

Каждый описанный выше пункт сложности *можно* решить вручную: структуры данных в OSaml есть, потоки тоже, таймеры в Unix доступны, голый код пишется. Но есть несколько

мест, где «можно вручную» означает *можно сделать так, что в проде это работать не будет*. miniFlink в каждом из них берёт на себя самую тонкую часть.

Event-time и watermark. Без явной модели событийного времени любая обработка с окнами и таймерами скатывается к *wall-clock*: «считаем средний RSSI за последнюю минуту по часам в нашем сервере». В шахте это даёт явные ошибки: пакет с `ts=88s` физически пришёл к сервису в `210s` (UDP + LTE-retry + очередь), и попадает в окно для текущей минуты, искажая её. miniFlink требует чтобы в потоке шли явные **Watermark**-события и каждый оператор окон работал по ним. Голый аналог пришлось бы писать самому, причём правильно: проверять ordering, перепроверять для каждого нового пайплайна, тестировать для каждого варианта late-семантики.

Retract как первоклассное событие. `allowed_lateness` в `window_agg_keyed` даёт нам *автоматический* retract: опоздавший пакет, который попадает в уже закрытое окно (в пределах `allowed_lateness`), вызывает **пересчёт** окна и эмиссию **Retract** старого значения плюс новый **Data**. Голый код имеет два пути: либо просто дропать опоздавшие пакеты (теряем данные ради простоты), либо пересчитывать без retract'ов (диспетчер видит мигание значений на дашборде, потому что окна *переоткрываются* молча). Третий правильный путь — сделать retract как явное событие — это и есть то, что библиотека реализует один раз и для всех пайплайнов.

Дедупликация по бизнес-правилу. UDP-шлюз в шахте ретранслирует пакеты при сомнениях — получаем дубли. Наивное “`HashSet` по сериализованным байтам” сломается мгновенно: шлюз обогащает пакет полем `received_at`, и идентичные по смыслу пакеты различаются на байтовом уровне. `Pipe.dedup` берёт правило вида `p.lamp ^ string_of_int p.ts` — ключевое поле бизнеса, не байты транспортного протокола.

State per-key + automatic cleanup. `Pipe.process_keyed` держит state per-key (по шахтёру), вызывает наш обработчик, поддерживает self-timer'ы для timeout- семантики (alert после двух минут тишины). На четырёх тысячах шахтёров голый код потребует свой `Hashtbl<lamp, state>`, ручную регистрацию таймеров, ручную очистку state после конца смены. Каждое из этого — источник багов; например, мы могли бы накопить state шахтёров прошлой смены и тек уть память. Библиотека предоставляет TTL-state из коробки.

Channel с backpressure. Грозовая помеха в радио — шлюз накапливает 30 секунд пакетов, потом резко выдаёт. Голый in-memory pipe без блокировки на upstream получает OOM, когда downstream не справляется. `Pipe.channel` в miniFlink имеет явный capacity — producer блокируется когда capacity исчерпан. Сеть-шлюз дольше держит запрос, но сервис *не падает*.

Объединение потоков с выравниванием watermark'а. Газовый пайплайн потребляет три источника — raw RSSI, окна локации, газ. У каждого свой event-time и свои watermark'и. Если их просто “склеить” (`Stream.append`), один поток обгонит другие, и состояние обогатится событиями “из будущего”. `Mf_event.union` выравнивает watermark по минимуму из подключенных потоков — ровно та семантика которая нужна, и которую *очень неприятно* писать руками каждый раз.

Параллелизация задним числом. Самое заметное. Изначальный ex07 написан в однопотоковом виде. Добавление параллельной обработки — это одна обёртка `Parallel.run_parallel_simple` вокруг готового пайплайна, без изменения бизнес-логики; см. раздел 5, где показано как одна и та же `Pipelines.median_rssi` ускорила в 4.17 раз на 8 воркерах после такой

обёртки. Голый код пришлось бы переписывать, разбираться с гонками, перепродумывать state-shared-vs-per-key. Здесь же исходник остался байт-в-байт тем же.

Тестируемость. Stream это просто `unit -> 'a Mf_event.t option`. Тест пайплайна — `Mf_event.of_list ~ts list` с явными временами, потом `Stream.to_list` результата, потом проверка инвариантов. Голый код, использующий системное время и сетевые сокеты, потребовал бы моков или медленных интеграционных тестов. У нас наш сложный газовый retract-пайплайн покрыт 7-ю сценариями за <10ms прогона.

Это не «библиотечная польза вообще». Это *конкретные* места, где наивный аналог даёт либо потерю данных, либо нестабильный дашборд, либо невозможность тестов. Каждое исправлять руками дорого; пакет работающих абстракций в **miniFlink** обходится дешевле.

1.4 Что есть готового в miniFlink

Кратко, чтобы не повторяться в дальнейших разделах — ниже полный inventory того, что мы используем в ex07:

- `Pipe.event_time ~lateness` — вставка watermark'ов в поток по event-time;
- `Pipe.dedup` — дедупликация по rule-функции;
- `Pipe.window_agg_keyed` с `allowed_lateness` — скользящие окна с retract'ами при опоздавших пакетах;
- `Agg.median`, `Agg.group_by`, `Agg.top_k_by` — набор комбинируемых агрегатов;
- `Pipe.process_keyed` — per-key state с self-timers (для FSM состояния шахтёра);
- `Mf_event.union` — объединение потоков с выравниванием watermark'a по минимуму;
- `Pipe.materialize` — сворачивание retract-богатого потока в финальную таблицу записей (для sink-a).

Чего *нет* — это бизнес-логики и доменных типов. Их пишем мы; структура ниже как раз и есть рассказ о том, как мы их написали.

1.5 Что увидим дальше

Статья идёт от внешнего наружу. В разделе 2 разбираем устройство входов: почему два независимых потока вместо одного, какие в них аномалии. В разделе 3 — три пайплайна по очереди: FSM состояния, скользящие окна с медианой и интерполяцией позиции, retract-обогащение газовых алертов. Раздел 4 собирает наблюдения о retract-семантике как о сквозном механизме, проходящем через все три пайплайна. Раздел 5 приводит измерения производительности (на машине автора-эксперимента и на пользовательском i7-8700 с OCaml 5) и описывает направленные оптимизации, которые мы делали. Раздел 6 — что отложено, что не делалось намеренно, и при каких условиях стоит реализовать недостающее.

2 Входы

В реальной системе входами выступают Kafka-топики, MQTT-брокеры, UDP-сокеты — что-то, что принимает байты с радио-сети шахты и дальше отдаёт их потребителю. В ex07 мы заменяем эту инфраструктурную часть на `Mock_source` — модуль, генерирующий *детерминированный* поток событий, неотличимый по форме от прода. Когда сервис переедет в minePASS-production, `Mock_source` меняется на `Kafka_source.read`, а *остальной код не трогается*. Это центральная идея ex07 — бизнес-логика не привязана к транспорту.

2.1 Два независимых потока, а не один

Первое и самое существенное решение — разнести RSSI-пакеты и газовые пакеты в два разных потока с разными типами.

```
module type S = sig
  val read      : unit -> packet      Mf_event.t Stream.t
  val read_gas  : unit -> gas_packet Mf_event.t Stream.t
  val stats     : unit -> string list
end
```

Почему не сделать единый `type telemetry = RSSI of packet | Gas of gas_packet` с одним потоком `read`? Соблазн есть: один поток — одна точка консьюинга, проще архитектура источника. Не делаем по трём причинам.

Физически разные устройства. В реальном фонаре RSSI измеряется радиомодулем, газ — отдельным модулем на ремне (часто с собственным `batterypack`’ом). Они шлют пакеты независимо, через разные радиостанции на шлюз. Если объединять их в источнике — надо объединять в шлюзе, а это *дополнительная* логика на устройстве с ограниченным CPU. Дешевле оставить два потока.

Разные частоты и разный жизненный цикл. RSSI — 15 секунд между пакетами, газ — 10. Объединение через `union`-тип создаст поток с *неравномерной* структурой («два RSSI, потом газ, потом RSSI...»), что усложняет понимание `timing`’а. Раздельные потоки имеют каждый свою регулярность.

Обогащение однонаправленное. Газовый пайплайн *потребляет* результат RSSI-пайплайна (координаты), но не наоборот. Если бы потоки были один, эта зависимость размывалась бы. А так в коде `Pipelines.gas_alerts` прямо принимает `~rssi`, `~locations`, `~gas` тремя именованными аргументами — видно что от чего зависит.

2.2 Структура RSSI-пакета

```
type packet = {
  lamp      : string;
  readings  : (string * float) list;
  voltage   : float;
  moving    : bool;
  sos       : bool;
  ts        : Time.t;
}
```

`lamp` — идентификатор фонаря, по нему идёт всё партиционирование (один шахтёр == один ключ). `ts` — event-time, миллисекунды от начала смены или от epoch (зависит от deployment).

`readings` — список пар (`beacon_id`, `rssi_dBm`). У шахтёра одновременно слышно от одного до пяти маяков (зависит от того, где он сейчас). Делать отдельную таблицу `packet` → `reading` с FK на пакет, как сделали бы в реляционной базе, нет смысла: пакет существует ровно как единое сообщение от устройства, и в потоке мы будем разворачивать его через `Pipe.flat_map` в плоский поток `reading`’ов.

`voltage`, `moving`, `sos` — метаданные о состоянии фонаря, обновляемые на каждом пакете. Это *не показания* в смысле RSSI, а отдельный класс данных (датчик ускорения, ADC

батарей, кнопка). Они здесь потому что физически живут в том же пакете: радиомодуль фонаря один раз в 15 секунд просыпается и отправляет всё разом. Логически они потом расходятся по разным пайплайнам.

2.3 Структура газового пакета

```
type gas_packet = {
  g_lamp : string;
  g_ts    : Time.t;
  g_co2   : float option;
  g_co    : float option;
  g_h2    : float option;
  g_ch4   : float option;
}
```

Газы все `option` осознанно. У разных моделей газового модуля разный набор сенсоров: бюджетный имеет только CO и CH₄, дорогой — все четыре. Если бы поля были `float` с магической константой “нет данных” (например, `-1.`), это проникало бы во все downstream пайплайны и рано или поздно пробралось бы в финальный отчёт как “CO = -1 ppm”. С `option` компилятор не даст забыть `match` на `None`.

CO и CO₂ выражаются в ppm; H₂ и CH₄ — тоже, для единообразия (хотя в шахтёрской документации их часто пишут в объёмных процентах: 1% CH₄ = 10 000 ppm).

2.4 Дрожание канала

`Mock_source.Default` специально создаёт реалистичный поток, включая два класса аномалий:

- **Дубли** (около 3% от трафика): тот же (`lamp`, `ts`) приходит дважды. Имитирует поведение at-least-once Kafka и retry-логику UDP-шлюза.
- **Опоздавшие пакеты** (тоже около 3%): пакет доходит до сервиса *позже* чем должен по event-time. Сдвиг сделан таким, что опоздание превышает размер скользящего окна (≥ 90 секунд) — это гарантирует, что опоздавший пакет попадает в *уже закрытое* окно и порождает `retract`.

Это не «для красоты тестирования». Это для того, чтобы бенчмарки *отражали прод-нагрузку*. На раннем этапе в репозитории был отдельный `Clean`-источник без аномалий, с обоснованием «бенчмарку нужна стабильная нагрузка». Это было ошибкой: на чистом потоке `dedup` ничего не дедуплицирует, `retract`-логика никогда не срабатывает, и измеренный `throughput` *не отражает* реальной работы. Источник был удалён, а в коммит-сообщении явно записано почему. Этот эпизод хорошо иллюстрирует принцип: бенчмарк должен включать *цену корректности*, иначе он показывает несуществующую скорость, которая исчезает на проде.

2.5 `Large_mine`: масштабная конфигурация

`Mock_source.Default` даёт 6 шахтёров с тщательно проработанными индивидуальными сценариями — этого достаточно для демонстрации и регрессии, но мало для измерения производительности. Параллельно есть `Mock_source.Large_mine` — настраиваемая большая шахта:

```
let default_config = {
  horizons          = 16;
  beacons_per_horizon = 64;
```

```

miners_per_horizon    = 256;
simulation_minutes     = 60;
step_seconds          = 15;
gas_period_seconds    = 10;
pct_no_packets        = 0.10;
pct_no_readings       = 0.05;
pct_no_motion         = 0.20;
pct_low_voltage       = 0.30;
pct_sos               = 0.01;
pct_gas_alerts        = 0.05;
pct_duplicates        = 0.03;
pct_late              = 0.03;
}

```

Шестнадцать горизонтов по 64 бикона = 1024 бикона; шестнадцать горизонтов по 256 шахтёров = 4096 шахтёров; час симуляции с шагом 15 секунд = около 949 000 RSSI-пакетов плюс газовые (шлются чаще). Это размер реальной большой шахты в Кузбассе.

Каждый шахтёр получает *детерминированный* сценарий через хеш своего ID — такие-то проценты попадают под каждый тип алерта, с непустыми пересечениями (один шахтёр может одновременно ронять батарейку и не двигаться). Детерминированность важна: повторный прогон бенчмарка даёт ровно те же события и ровно тот же результат, можно сравнивать через `git bisect`.

Биконы на горизонте. Маяк связи слышен только на *своём* горизонте — это упрощение, но не очень сильное: на практике межгоризонтная связь через породу настолько слабая, что её обычно специально подавляют, чтобы триангуляция не запуталась. Каждый шахтёр на своём горизонте слышит до пяти ближайших биконов по сеточному расположению. RSSI вычисляется детерминированно из расстояния по формуле $RSSI = -40 - 20 \log_{10}(d)$, и обращается обратно в расстояние в пайплайне локации — так `Large_mine` полностью внутренне-согласован.

Дальше мы переходим к самим пайплайнам.

3 Три пайплайна

Три пайплайна образуют ядро сервиса. У каждого свой набор операторов и свои дизайнерские решения, но между ними виден общий стиль: все три потребляют один и тот же `packet`-поток (или его производные), все три используют event-time, у всех трёх результат идёт в sink-обвязку, которая отображает или публикует данные.

Разбираем по одному. Самый простой по семантике, но самый интересный архитектурно — `connectivity_alerts`.

3.1 connectivity_alerts: три слоя FSM

Этот пайплайн ловит пять диагностических состояний шахтёра:

- **No_packets** — пакеты от фонаря перестали приходить (> 2 мин тишины);
- **No_readings** — пакеты идут, но `readings` пустой (> 5 мин не слышит ни одного маяка);
- **No_motion** — датчик ускорения молчит (> 2 мин без движения);
- **Low_voltage** — напряжение батареи ниже 3,5 В, держится дольше 2 минут (с гистерезисом возврата 3,7 В);

- **Sos** — кнопка SOS нажата (мгновенно, без таймера).

Любая из четырёх первых тревог снимается, когда условие перестаёт выполняться (для напряжения — с гистерезисом, об этом ниже).

Соблазн монолита. Самое прямое решение — один большой `process_keyed.on_event` с per-key state, в котором аккумулируем «время последнего пакета, время последнего движения, текущее напряжение, флаг что был ли уже `Low_voltage`». На каждом пакете обновляем state, смотрим что изменилось, эмитим алерты.

Это работает, но плохо читается и плохо тестируется. На каждом пакете мы одновременно делаем три разных вещи: **запоминаем факты** (когда последний пакет), **принимаем решения** (нужно ли алертить), **регистрируем таймеры** (когда проверить тишину снова, если сейчас пакетов нет). Все три смешаны в одной функции, и поправить поведение для одного типа алерта означает читать весь файл.

Разделение на три слоя. Поэтому Pipelines раскладывает FSM на три модуля:

- **Last_seen** — хранилище фактов;
- **Fsm** — чистые функции «факты → алерт»;
- **Self_timer** — регистрация таймеров.

`process_keyed` склеивает их в общий пайплайн, но каждый модуль изолирован и тестируем независимо.

3.1.1 Слой 1: Last_seen

Просто запись того, что мы знаем о шахтёре в данный момент.

```
type t = {
  mutable last_pkt_ts      : Time.t;      (* any packet *)
  mutable last_reading_ts : Time.t;      (* packet with non-empty
    readings *)
  mutable last_motion_ts  : Time.t;      (* packet with moving=true *)
  mutable last_voltage    : float;
  mutable last_voltage_ts : Time.t;
  mutable low_v_since     : Time.t option;
  mutable in_low_v        : bool;        (* for debounce + hysteresis *)
}
```

Метод `record` обновляет нужные поля по приходящему пакету. В нём *нет* логики «что эмитить» — только запоминание.

Этот слой решает проблему «когда таймер сработал в момент молчания, у нас нет пакета чтобы взять из него факты» — факты уже лежат в `Last_seen`.

3.1.2 Слой 2: Fsm

Чистые функции от `Last_seen` к опциональному алерту:

```
let no_packets seen ~now =
  if now - seen.last_pkt_ts > no_packets_threshold
  then Some (No_packets (seen.lamp, seen.last_pkt_ts))
  else None
```

```

let no_motion seen ~now = ...
let no_readings seen ~now = ...
let low_voltage seen ~now = ...

(* Aggregate: at what nearest moment in time should we
   re-check the state, assuming no new packet arrives? *)
let next_check seen = ...

```

В этом слое нет state и нет I/O — любая функция здесь принимает `Last_seen.t` и возвращает чистое значение. Тесты пишутся тривиально: «вот snapshot, вот ожидаемый алерт».

next_check как сердце таймера. Самая интересная функция здесь — `next_check`. Она отвечает на вопрос: «если сейчас *не* придёт нового пакета, в какой ближайший момент состояние может измениться?» Например, если последний пакет был в 100с и порог тишины — 2 минуты, то изменение наступит в 220с (100 + 120). Если последнее движение было в 95с — то в 215с. И так далее. `next_check` берёт минимум по всем «пробудкам» и возвращает один timestamp.

Это нужно `Self_timer`'у, чтобы знать, на какой момент будить пайплайн.

3.1.3 Слой 3: Self_timer

`process_keyed` в `miniFlink` имеет встроенную поддержку self-timers через `ctx.set_timer`. `Self_timer` тонко оборачивает её, добавляя одну инвариантность: на каждом ключе живой может быть *максимум один* таймер. Когда приходит новый пакет и `Fsm.next_check` даёт новое время будильника, старый таймер отменяется и регистрируется новый. Это уберёгает от накопления старых таймеров (особенно опасно для долгоживущих шахтёров).

```

let reschedule t ctx ~target =
  match t.armed with
  | Some old when old = target -> ()          (* same timer *)
  | _ ->
    (* Old timer will be ignored on fire -- we changed 'armed',
       so when it fires the handler sees it's stale. *)
    ctx.Pipe.set_timer target Pipe.Wallclock;
    t.armed <- Some target

```

3.1.4 Склейка через process_keyed

```

let connectivity_alerts source =
  source
  |> Pipe.event_time ~lateness:(seconds 1)
  |> Pipe.process_keyed (module ByLamp)
    ~init:make_state
    ~on_event:(fun ctx key st p ->
      (* Layer 1: remember facts *)
      (match Last_seen.record st.seen ~p with
      | 'Sos_pressed -> ctx.emit (Sos (key, p.ts))
      | 'Quiet -> ());
      (* Layer 2 + emission *)
      check_and_emit ctx key st ~now:p.ts;
      (* Layer 3: schedule the next check *)
      Self_timer.reschedule st.timer ctx
        ~target:(Fsm.next_check st.seen))
    ~on_timer:(fun ctx key st t_kind ->

```

```

Self_timer.consumed st.timer;
check_and_emit ctx key st ~now:t;
let next = Fsm.next_check st.seen in
if next > t && next < max_int then
    Self_timer.reschedule st.timer ctx ~target:next)

```

Двадцать строк кода, и каждая строка делает ровно одну вещь. Это прямой эффект разделения: ни одна функция в этом пайплайне не смешивает «помню что было» с «решаю что делать».

Гистерезис напряжения. Один из неприметных, но важных моментов — `Low_voltage` имеет *два* порога: `low_voltage_threshold = 3.5` (вход в подозрение) и `voltage_ok_threshold = 3.7` (выход обратно). Это *гистерезис*: вошёл в `Low_voltage` при $V < 3,5$, вышел только при $V > 3,7$. Между ними — *мёртвая зона*, в которой состояние не меняется.

Без гистерезиса колебания напряжения около 3,5 В (нормальное поведение под нагрузкой фонарика) дали бы *дребезг алертов* — `Low_voltage`, `Voltage_ok`, `Low_voltage`, `Voltage_ok` — много раз в минуту. Диспетчер увидел бы хаос. Один из тех мелких моментов, которые отличают `workable` систему от непригодной.

Дополнительно есть `voltage_debounce = 2` минуты: алерт `Low_voltage` эмитится не сразу как напряжение упало ниже 3,5, а только если оно *держится* там 2 минуты подряд. Это уже не гистерезис, это `debounce` — защита от кратковременных просадок напряжения (включение фонаря на полную мощность даёт мгновенный провал).

Почему такой подход универсален. Каждый раз, когда в проде встаёт задача «детектировать состояние из временного ряда событий», возникают те же три проблемы: а) надо помнить факты; б) надо принимать решения о них; в) надо реагировать на молчание. `Three-layer split` здесь не специфичен для шахты или для `miniFlink` — он будет применим к любой подобной задаче (мониторинг IoT-устройств, `heartbeats` веб-сервисов, `dropped sessions` в чатах). В этом примере он реализуется в идиомах `OCaml + process_keyed`, в `Java + Flink KeyedProcessFunction` был бы тот же `split` в тех же трёх классах.

Дальше переходим к пайплайну локации — архитектурно проще (один длинный `pipe` из операторов), но с насыщенной алгоритмикой: скользящие окна с `retract`'ами, медиана по биконам, `top-2`, и линейная интерполяция координат.

3.2 median_rssi: скользящие окна и интерполяция координат

Задача пайплайна — для каждого шахтёра выдавать «топ-2 маяков с самой сильной медианной RSSI» в скользящем 60-секундном окне с шагом 5 секунд, а из них приближённо вычислять координаты шахтёра в двумерной плоскости горизонта.

Архитектурно это *линейный* `pipe` из операторов; интересна не структура, а каждый отдельный шаг и почему он выглядит так.

3.2.1 Шаги пайплайна

1. dedup по бизнес-ключу.

```

|> Pipe.dedup (module ByLamp)
  ~rule:(fun p -> string_of_int p.ts)
  ~cooldown:(seconds 120)

```

`Pipe.dedup` держит per-key cache недавно виденных rule-значений и фильтрует повторы. Ключ дедупликации — `(lamp, ts)`, выражаемый как `string_of_int p.ts` в пределах ключа `p.lamp`. Если пакет от M1 с `ts=88s` пришёл дважды (at-least-once Kafka, UDP retry), мы дедуплицируем по бизнес-ключу *пакета* (`lamp + event-time`), а не по байтовому содержимому сетевого сообщения — которое шлюз вполне мог обогатить полем `received_at` перед ретрансляцией.

`cooldown 120s` означает: дедуплицируем дубли в пределах двух минут. Если тот же ключ придёт через 3 минуты — это уже не дубль, а что-то странное (битый clock на устройстве?), но пропускаем дальше, не теряя данные.

2. flat_map: разворот в reading'и.

```
|> Pipe.flat_map (fun p ->
  List.map (fun (b, rssi) ->
    { r_lamp = p.lamp; r_beacon = b;
      r_rssi = rssi; r_ts = p.ts })
  p.readings)
```

Каждый пакет имеет от одного до пяти reading'ов. `flat_map` разворачивает один пакет в несколько событий, каждое — запись вида “шахтёр X слышал маяк Y с уровнем Z в момент T”. Дальше работаем с плоским потоком, а не с пакетами.

Почему это правильно: для медианы по бикону нам нужно *сгруппировать* все RSSI данного маяка отдельно, что естественно делается на плоских записях. Если бы оставили пакеты, `group_by` пришлось бы делать по `packet.readings`, и агрегаты усложнились бы.

3. event_time с lateness.

```
|> Pipe.event_time ~lateness:(seconds 1)
```

Вставка watermark'ов на основе event-time каждого reading'a. Параметр `lateness:1s` означает: «watermark отстаёт от максимального виденного event-time на 1 секунду». Это *out-of-order tolerance* между близкими во времени событиями: если у нас сейчас прошло событие с `ts=100s`, watermark поднимется только до 99с, давая 1 секунду на “последние опоздавшие” до того как окно закроется.

Почему именно 1 секунда. В нашем сценарии reading'и сгенерированы из одного пакета с одинаковым `ts` — они синхронны. Out-of-order возникает только между пакетами от разных шахтёров, доставленными через сеть. Эмпирически разброс доставки для не-аномальных пакетов укладывается в 1с. Если поставить больше (скажем, 60с) — окна закрываются медленнее, дашборд лагает на минуту. Меньше — теряем близко-опоздавшие. Это типичный out-of-order trade-off; 1с в нашем сценарии нормально, в шахтах с очень плохим радио может потребоваться 5–10с.

Сама обработка *сильно* опоздавших пакетов (более 60с) лежит на `allowed_lateness` следующего оператора — они переоткрывают окно через `retract`.

4. window_agg_keyed: главный оператор.

```
|> Pipe.window_agg_keyed
  ~by:(fun r -> r.r_lamp)
  ~allowed_lateness:(seconds 60)
  (Pipe.sliding (seconds 60) (seconds 5))
  Agg(...)
```

Здесь несколько параметров:

- `by` — ключ партиционирования (шахтёр);
- `sliding (60s) (5s)` — окно длиной 60с с шагом 5с, то есть событие $t = 30с$ попадает в окна $[-30, 30)$, $[-25, 35)$, $[-20, 40)$..., $[30, 90)$. Двенадцать окон на событие;
- `allowed_lateness 60s` — если опоздавший пакет попадает в *уже закрытое* окно, но не более чем на 60с прошлое — окно **переоткрывается**, агрегат пересчитывается, эмитится `Retract` старого результата плюс `Data` нового. Пакеты опоздавшие сильнее — идут в side output (мы их не собираем; в проде шли бы в DLQ);
- `Agg.t` — комбинаторно-собираемый агрегат, см. ниже.

Это *единственный* stateful оператор всего пайплайна. Состояние — хеш-таблица «окно → список значений (или сложенный аккумулятор)», очищается автоматически при закрытии окна (или оставляется на время `allowed_lateness`).

5. Двухуровневая агрегация через Agg.

```
Agg.(
  group_by
    ~key:(fun r -> r.r_beacon)
    ~inner:(median (fun r -> r.r_rssi))
  |> map (fun per_beacon ->
    per_beacon
    |> List.filter_map (fun (b, med) ->
      match med with
      | Some m -> Some (b, m)
      | None -> None)
    |> Agg.run (top_k_by 2 ~by:snd)))
```

Внутри одного окна шахтёра M1 у нас может быть, например, 8 reading'ов от B1, 12 от B2, 5 от B3. `group_by key:beacon` группирует их по маяку. `~inner: median` — агрегат, применяемый к каждой группе: для B1 даёт medianX, для B2 — medianY, для B3 — medianZ. На выходе списка пар $[(B1, X); (B2, Y); (B3, Z)]$.

Дальше `map` превращает этот список в top-2 через `top_k_by 2`. У нас два уровня агрегации — сначала медиана внутри маяка, потом ranking между маяками — собранные *декларативно* из библиотечных кубиков, без рукописных циклов.

3.2.2 Линейная интерполяция координат

Имея top-2 маяков с известными координатами и медианной RSSI до каждого, мы хотим приблизительно найти координаты шахтёра в плоскости горизонта.

От RSSI к расстоянию. В Large_mine RSSI генерируется по формуле $RSSI = -40 - 20 \log_{10}(d)$. Инверсия:

$$d = 10^{\frac{-RSSI-40}{20}}$$

```
let distance_from_rssi (rssi : float) : float =
  10. ** ((-. rssi -. 40.) /. 20.)
```

В реальной шахте этот калибровочный коэффициент зависит от частоты радио, влажности воздуха, рельефа тоннеля, и в проде калибруется эмпирически на каждом горизонте. У нас вшито в код — демо.

Интерполяция между двумя маяками. Имея A с координатами (x_a, y_a) и расстоянием d_a , B с координатами (x_b, y_b) и расстоянием d_b , точку на отрезке *AB* пропорционально

расстояниям:

$$t = \frac{d_a}{d_a + d_b}, \quad \text{pos} = (1 - t) \cdot A + t \cdot B$$

```
let interpolate_position ~find_beacon (top2 : (string * float) list)
=
match top2 with
| (b_a, rssi_a) :: (b_b, rssi_b) :: _ ->
  (match find_beacon b_a, find_beacon b_b with
   | Some (xa, ya, ha), Some (xb, yb, _) ->
     let d_a = distance_from_rssi rssi_a in
     let d_b = distance_from_rssi rssi_b in
     let total = d_a +. d_b in
     if total = 0. then Some (xa, ya, ha)
     else
       let t = d_a /. total in
       let x = (1. -. t) *. xa +. t *. xb in
       let y = (1. -. t) *. ya +. t *. yb in
       Some (x, y, ha)
   | _ -> None)
| _ -> None
```

Точность. Низкая. Шахтёр почти никогда не лежит точно на отрезке между двумя биконами — он в 2D-плоскости. Без 3-го маяка позиция геометрически не определяется однозначно: пересечение двух окружностей с центрами в биконах даёт две точки симметрично относительно линии AB , наш алгоритм просто выбирает середину между ними.

В реальной production-системе это нужно делать *трилатерацией* на 3+ маяках: метод наименьших квадратов на пересечении трёх (и более) окружностей. Существенно сложнее в коде (нужен solver нелинейной системы), но даёт реалистичную позицию.

Делаем интерполяцию намеренно: на ex07 это *демо*, и интерполяция показывает идею «координаты вычисляются из чего-то, у нас есть что-то». Газовому пайплайну для обогащения алертов важнее *наличие хоть какой-то позиции*, чем её точность — “приблизительно где” лучше для спасателя чем “неизвестно где”.

3.2.3 Single-beacon fallback

Что делать если в reading’ах шахтёра *один* маяк или ноль? `interpolate_position` вернёт `None`. Но для газового пайплайна это плохо: первый алерт мы можем эмитить вообще без позиции, и хотелось бы хоть какую-то указать.

`single_beacon_position` — вспомогательная функция: берёт reading с самым сильным RSSI и возвращает координаты *этого маяка*. То есть «шахтёр где-то рядом с B1, точнее сказать не можем». Точность ниже чем у интерполяции, но это лучше чем `None`.

```
let single_beacon_position ~find_beacon (readings : (string * float)
list) =
match readings with
| [] -> None
| rs ->
  let (b, _) = List.fold_left
    (fun ((_, best) as acc) ((_, r) as cur) ->
      if r > best then cur else acc)
    (List.hd rs) (List.tl rs) in
  find_beacon b
```

Эта функция используется в газовом пайплайне (раздел 3.3) для обогащения координат *до* того как первое окно RSSI закрылось — сразу после первого raw RSSI-пакета шахтёра.

3.2.4 Собирая всё вместе

Финальный пайплайн:

```
let median_rssi ?(find_beacon = Domain.beacon_coords)
  (source : packet Mf_event.t Stream.t)
  : location Mf_event.t Stream.t =
source
|> Pipe.dedup (module ByLamp)
  ~rule:(fun p -> string_of_int p.ts)
  ~cooldown:(seconds 120)
|> Pipe.flat_map (fun p -> ... (* expand to readings *))
|> Pipe.event_time ~lateness:(seconds 1)
|> Pipe.window_agg_keyed ~by:(fun r -> r.r_lamp)
  ~allowed_lateness:(seconds 60)
  (Pipe.sliding (seconds 60) (seconds 5))
  Agg.(group_by ~key:... ~inner:median |> map top_k_by_2)
|> map_to_location_with_position ~find_beacon
```

Шесть операторов, каждый делает ровно одну вещь. Можно прочитать сверху вниз и понять полный жизненный цикл события. Это и есть обещание stream processing fluent-DSL: пайплайн читается как описание *бизнес-преобразования*, без runtime-деталей про state и таймеры.

Что эмитит этот пайплайн. Поток событий `location Mf_event.t: Data` при закрытии окна, `Retract` плюс новый `Data` при опоздавшем пакете в `allowed_lateness`. `Watermark`'ы продвигаются по event-time.

Sink через `Pipe.materialize` сворачивает поток в *финальную таблицу* «(шахтёр, конец окна) → location» — `retract`'ы автоматически отменяют старые значения. Это позволяет диспетчерскому пульту показывать *актуальную картину*, не флудя на каждом обновлении.

Дальше — самый интересный пайплайн, газовый. У него три потока на входе, и `retract`-семантика не из коробки оператора окон, а написана вручную.

3.3 gas_alerts: три потока и retract вручную

Это самый интересный из трёх пайплайнов, и единственный из них, в котором `retract`-семантика реализована *вручную*, а не встроенным оператором окна. По дороге заодно отвечаем на исходный вопрос задачи: *если пакеты с координатами приходят позже газа, обновятся ли уже эмитированные алерты?* Ответ — да, через механизм описанный ниже.

3.3.1 Что хотим

Газовый пакет приходит *отдельно* от RSSI-пакета и сам по себе не знает координат шахтёра. Когда в пакете обнаружено превышение порога ($\text{CO} > 50 \text{ ppm}$, $\text{CH}_4 > 5000 \text{ ppm}$, и т.д.) — мы хотим:

- эмитить алерт *немедленно*, не дожидаясь координат (потому что безопасность важнее красоты);
- включить в алерт координаты если они *известны* (из любого источника: грубо из последнего RSSI-пакета, точно из последнего закрытого окна локации);

- *обновить* ранее эмитированный алерт когда координаты становятся доступны или уточняются — через `retract` старого и `emit` нового;
- не флудить диспетчера на стабильных показаниях — одно превышение давать один *живой* алерт пока шахтёр в опасной зоне; обновлять алерт только при существенных изменениях.

Это явно *stateful* обработка по ключу (шахтёр + газ), с *временной зависимостью* между двумя независимыми потоками (газ и позиция). Кажется задачей для `stream join`. Не делаем `join` по двум причинам.

3.3.2 Почему не `window join`

В Flink-DSL есть оператор `join` с временным окном: *смастить* событие из левого потока с событием из правого потока, если их `event-time` различается не более чем на Δ . Применить к нашей задаче:

- **Задержка алерта.** Чтобы сделать `join`, нужно *дождаться* пары — газа и позиции. Если газа нет — ждём. Если позиции нет — ждём. Окно длиной 30с означает *до 30 секунд задержки* газового алерта. В шахте при критическом СО это непригодно: алерт нужен сразу, координаты или нет.
- **Дубли при разных частотах.** Газ шлётся каждые 10с, окно локации закрывается каждые 5с. Каждый газовый пакет матчится *несколькими* `location`-окнами в пределах временного окна `join`'а. Один газ + 3 близких `location`-окна = 3 алерта вместо одного. Можно фильтровать “по ближайшему”, но это уже не симметричный `join`.

То что мы делаем семантически — это *temporal left-join* с особенностью «брать последнее известное, не ждать пары». В Flink-терминах это `KeyedCoProcessFunction` с `ValueState<Position>`: газ — `driver`, позиция — `side input`, обновляется в `state` но не блокирует газ.

3.3.3 Три потока на входе

```
type gas_input =
  | Position_from_window of location
  | Position_from_raw    of packet
  | Gas                  of gas_packet
```

Три варианта одного `union`-типа. Первые два — разные источники позиции:

- **Position_from_window** — результат `median_rssi`, точная позиция из закрытого 60-секундного окна. *Отстаёт* от реального времени (до минуты), но точнее (медиана RSSI, интерполяция между двумя биконами).
- **Position_from_raw** — грубая позиция от `raw` RSSI-пакета через `single_beacon_position`. Обновляется *мгновенно* на каждом пакете (раз в 15 секунд), точность — координаты ближайшего бикона (где-то на сфере радиуса d).

Зачем нам *оба*? Чтобы алерты имели координаты с самого первого момента работы сервиса. Без `Position_from_raw` первый газовый алерт после старта смены не имеет координат целую минуту — пока не закроется первое RSSI-окно. С `raw-fallback` — координаты появляются в первом же газовом алерте после первого RSSI-пакета (то есть в течение 15 секунд).

Три потока сливаются через `Mf_event.union`, выравнивающий `watermark` по минимуму:

```
let s_loc = locations |> Pipe.map (fun l -> Position_from_window l)
in
```

```
let s_raw = rssi      |> Pipe.map (fun p -> Position_from_raw p) in
let s_gas = gas       |> Pipe.map (fun g -> Gas g) in
let unioned = Mf_event.union (Mf_event.union s_loc s_raw) s_gas
```

Watermark-выравнивание не игрушка. `Mf_event.union` ждёт пока все подключённые потоки продвинут watermark, и только тогда продвигает свой. Без этого один из потоков обогнал бы другие и события «из будущего» обогащали бы state до того как у нас есть «прошлые» события другого потока. Например, газовый пакет $t = 100s$ обработался бы до того как позиционный пакет $t = 80s$ обновил state — алерт ушёл бы без координат, хотя они уже *были известны*. Union с min-watermark гарантирует строгий event-time порядок.

3.3.4 Состояние рег-шахтёр

```
type gas_state = {
  mutable position : (float * float * int) option;
  active : (gas, active_alert) Hashtbl.t;
}
and active_alert = {
  aa_level      : gas_level;
  aa_ppm        : float;
  aa_position   : (float * float * int) option;
}
```

`position` — последняя известная координата шахтёра (перезаписывается при обновлении из любого источника позиции). `active` — хэш-таблица «газ → что мы последний раз эмитили по нему», по одному входу на каждый активный алерт. Когда шахтёр в норме по CH_4 , но активен по CO — в таблице один вход (`Gas_CO, ...`).

`active_alert` помнит *что было эмитировано*: уровень, ppm, позиция на момент эмиссии. Это нужно потому что retract должен *в точности повторить* старое событие (включая позицию которая могла измениться) — иначе sink не поймёт что retract относится к тому самому раннему алерту.

3.3.5 Шесть случаев на одном газовом пакете

Когда приходит газовый пакет, для каждого газа в нём (CO , CO_2 , H_2 , CH_4 — те что измерены) выполняется `process_one_gas`. Логика разбита на шесть случаев в зависимости от пары (текущий уровень опасности, был ли активный алерт):

```
match current_level, previous_alert with
```

Случай 1: норма, не было алерта. Ничего не делаем.

```
| None, None -> []
```

Случай 2: норма, был алерт — ситуация улучшилась. Эмитим retract старого `Gas_alert` + новый `Gas_resolved`:

```
| None, Some old ->
  Hashtbl.remove st.active g;
  [ Mf_event.retract (Gas_alert {...old...}) ts;
```

```
Mf_event.data      (Gas_resolved {...})      ts ]
```

`Gas_resolved` — отдельный конструктор. По нему sink понимает что тревога *снята* (а не просто исчезла из ленты). Параллель с `Voltage_ok` в `connectivity` — диспетчер видит явное событие «теперь в норме».

Случай 3: превышение, не было алерта — новый алерт.

```
| Some level, None ->
  Hashtbl.add st.active g { aa_level=level; aa_ppm=ppm;
                           aa_position=st.position };
  [ Mf_event.data (Gas_alert {level; ppm; position=st.position}) ts ]
```

Записываем в `active` что эмитили, эмитим `Data`.

Случаи 4, 5, 6: был алерт, всё ещё превышение.

```
| Some level, Some old ->
  let level_changed      = level <> old.aa_level in
  let ppm_changed        = ppm_changed_significantly old ppm in
  let position_changed   = st.position <> old.aa_position in
  if not (level_changed || ppm_changed || position_changed) then []
  else (* retract old + emit new *)
```

Три проверки:

- `level_changed` — Warning стал Critical или наоборот;
- `ppm_changed` — изменение более чем на 20%;
- `position_changed` — координаты обновились (главный случай для исходного вопроса задачи).

Если ни одно из условий не выполнено — ничего не эмитим (*sticky* семантика, защита от спама). Иначе — `retract` старого + `emit` нового.

Порог 20% на ppm. Сделан конфигурируемым через `Domain.gas_ppm_significant_change`. Меньше — спам обновлений на каждом колебании датчика. Больше — диспетчер не видит роста опасности. 20% — эмпирический баланс.

3.3.6 Refresh при обновлении позиции

Когда приходит `Position_from_window` или `Position_from_raw`, мы обновляем `st.position`. Но это в одиночку *не* эмитит обновлений уже-активных алертов. Для них нужен явный проход:

```
let refresh_alerts_for_new_position st lamp ts =
  Hashtbl.iter (fun g old ->
    if st.position <> old.aa_position then begin
      (* retract old + data with new position *)
      Queue.push (Mf_event.retract (Gas_alert {...old...}) ts)
        out_buf;
      Queue.push (Mf_event.data      (Gas_alert {...new...}) ts)
        out_buf;
      Hashtbl.replace st.active g { old with aa_position =
        st.position }
    end
  ) st.active
```

Этот проход вызывается *сразу после* обновления `st.position`. Все активные алерты текущего шахтёра получают `retract + emit` с новой позицией. Это и есть *обновление состояния газа уже с координатами* из исходного вопроса задачи.

Порядок приоритетов источников. На `Position_from_window` обновляем безусловно — точная позиция всегда заменяет грубую. На `Position_from_raw` обновляем *только если позиции ещё нет*:

```
| Position_from_raw pkt ->
| (match st.position, single_beacon_position pkt.readings with
|   None, (Some _ as new_pos) ->
|     st.position <- new_pos;
|     refresh_alerts_for_new_position st pkt.lamp pkt.ts
|   _ -> ())
```

Иначе грубая позиция от raw RSSI *переписала* бы более точную позицию от закрытого окна — и алерты флар'али бы между точной и грубой каждые 15 секунд.

3.3.7 Почему руками, а не `process_keyed`

`Pipe.process_keyed` — идиоматичный способ построить stateful per-key оператор. Логично было бы использовать его и здесь. Но не получилось.

`process_keyed.on_event` принимает callback с `ctx`, у которого есть `ctx.emit : 'out -> unit`. То есть пользователь эмитит *значение* типа `'out`, а runtime сам оборачивает его в `Mf_event.Data (v, t)`. Это удобно когда мы хотим эмитить только данные.

Но нам нужны `Mf_event.Retract`. Через `ctx.emit` их не выпустить — API не предусмотрен.

Это не баг библиотеки, а сознательное ограничение `process_keyed`: подавляющее большинство stateful операторов эмитит только `Data`, а `retract`'ы порождают *операторы окон* (через `allowed_lateness`). Газовый алерт — *нетипичный* случай, где пользовательский код сам решает когда `retract`.

Поэтому `gas_alerts` написан *вручную*: per-lamp `Hashtbl` для state, `Queue` для накопления исходящих событий, рекурсивный pull по upstream-потoku.

```
let states : (string, gas_state) Hashtbl.t = Hashtbl.create 64 in
let out_buf : gas_alert Mf_event.t Queue.t = Queue.create () in
let process_input = function ... in
let rec next () =
  if not (Queue.is_empty out_buf) then Some (Queue.pop out_buf)
  else match unioned () with
  | None -> None
  | Some (Mf_event.Watermark wm) -> Some (Mf_event.Watermark wm)
  | Some (Mf_event.Retract _) -> next ()
  | Some (Mf_event.Data (input, _ts)) ->
    process_input input; next ()
in next
```

Двадцать строк, и каждая делает ровно одно: либо выдаёт накопленное из буфера, либо подтягивает следующее событие из upstream, либо обрабатывает событие и рекурсивно идёт дальше.

Что это значит для библиотеки. Если бы такой паттерн встречался часто, имело бы смысл расширить `process_keyed` до варианта где `ctx.emit` принимает `Mf_event.t`. Пока это единственный случай, и руками — приемлемо. Архитектурное правило: когда у библиотеки нет ровно нужного API, можно реализовать *по своему*; main concern — читаемость и тестируемость, а не “всё через одну функцию”.

3.3.8 Главный сценарий: позиция приходит позже газа

Соберём всё вместе на примере шахтёра M1 со сценарием из `Mock_source.Default`:

1. $t = 10s$ — газовый пакет M1 с $CO = 120$ ppm (Critical). `st.position = None`. Эмитим `Gas_alert(level=Critical, ppm=120, position=None)`. `active[Gas_CO]` запомнила это.
2. $t = 15s$ — первый RSSI-пакет M1 с `reading`’ами от B1, B2, B3. `Position_from_raw`. `st.position` была `None`, `single_beacon_position` даёт координаты B1 (сильнейший `reading`). `st.position` обновлена. `refresh_alerts_for_new_position` проходит по `active`: видит `Gas_CO` с `position=None`, `position` обновилась → эмитим `Retract(old alert, position=None) + Data(new alert, position=B1’s coords)`.
3. $t = 20s$ — ещё один газовый пакет $CO = 119$ ppm. Critical тот же, ppm в пределах 20% (119 vs 120), `position` та же. Все три проверки `false` — ничего не эмитим.
4. $t = 60s$ — первое RSSI-окно $[0, 60)$ закрывается. M1 имел в нём `reading`’и от B1 и B2 (как минимум). `median_rssi` эмитит `location` с `loc_position` равной интерполяции B1-B2. `Position_from_window` обновляет `st.position` — безусловно (точная позиция важнее грубой). `refresh_alerts_for_new_position` снова: `Retract` старого алерта (с грубой позицией B1) + `Data` с новой позицией.

После этого алерт *остаётся живым* с актуальной позицией. Если координаты дальше не меняются (`sliding` окна выдают ту же интерполяцию) — больше ничего не эмитится. Если меняются — ещё один `retract + emit`. Если CO падает ниже 50 ppm — `retract + Gas_resolved`.

Что видит диспетчер. Sink через `Pipe.materialize` сворачивает `retract`’ы. На дашборде в любой момент видна *одна* запись на (M1, CO) с *текущим* ppm и *актуальной* позицией. Промежуточные `retract`’ы под капотом обеспечивают что эта запись правильная — но на экран попадает только финальная картина. Это и есть ценность `retract`-семантики — диспетчер видит правду, не флудя истории.

На этом разбор трёх пайплайнов закончен. В следующем разделе обобщаем `retract` как *сквозной механизм* проходящий через все три, и объясняем как sink его материализует.

4 Retract как сквозной механизм

Этот раздел ещё не написан.

5 Производительность

Этот раздел ещё не написан.

6 Выводы

Этот раздел ещё не написан.